

CESUR CARTUJA — SEVILLA

Ciclo Formativo de Grado Superior, Desarrollo de Aplicaciones Multiplataforma (DAM)

PROYECTO FIN DE MÓDULO INTERMODULAR

NutriFit

Sistema de seguimiento nutricional y deportivo

Autor:

Antonio Manuel Fresco Gómez

Curso: 2.º DAM · Año académico 2025/2026

<https://github.com/AntManFresc2006/NutriFit>

Mayo de 2026

Índice

1. Introducción y contexto

1. Problema que aborda el proyecto
2. Objetivo del proyecto
3. Alcance del sistema
4. Arquitectura y tecnologías
5. Organización de la memoria

2. Estado del arte y marco teórico

3. Tecnologías y herramientas

4. Análisis de requisitos

5. Diseño del sistema

6. Implementación

7. Pruebas

8. Seguridad

9. Conclusiones y líneas futuras

10. Bibliografía

Anexo A — Guía de puesta en marcha

Anexo B — Referencia de endpoints de la API

1. Introducción y contexto

El seguimiento de la alimentación es una práctica habitual en contextos de salud, deporte y gestión del peso. Registrar qué se come, cuánto y cuándo da a una persona información objetiva sobre su ingesta calórica y su distribución de macronutrientes a lo largo del día. Sin ese registro, cualquier ajuste dietético se basa en estimaciones.

Existen herramientas comerciales que cubren esta necesidad, pero su complejidad interna queda opaca al desarrollador que quiera aprender de ellas. NutriFit parte de la misma necesidad funcional y la aborda como ejercicio de ingeniería: construir desde cero un sistema multicapa que gestione alimentos, registre comidas, calcule resúmenes nutricionales diarios, integre inteligencia artificial para evaluar la dieta, registre ejercicios y estime el gasto energético del usuario, con decisiones técnicas razonadas y documentadas en cada paso.

1.1 Problema que aborda el proyecto

El sistema responde a una necesidad concreta: que un usuario pueda registrar los alimentos que consume durante el día, organizarlos por comidas, obtener un resumen del total calórico y de macronutrientes, y recibir una evaluación personalizada de su dieta basada en inteligencia artificial.

El problema no es complejo en su enunciado, pero su implementación involucra decisiones no triviales: cómo modelar la relación entre comidas e ítems, cómo calcular los valores nutricionales sin mover datos innecesariamente entre capas, cómo gestionar sesiones con logout real, cómo integrar modelos de lenguaje sin acoplar la lógica de negocio a un proveedor concreto, o cómo garantizar que el esquema de base de datos es reproducible en cualquier entorno. Esas decisiones son el centro del trabajo.

1.2 Objetivo del proyecto

El objetivo principal es diseñar, implementar y documentar un sistema completo de seguimiento nutricional con tres interfaces de usuario: una aplicación web (React SPA), un cliente de escritorio (JavaFX) y una API REST como capa central. La versión web está desplegada en producción y es accesible públicamente.

El objetivo no es construir una aplicación comercial lista para escalar a millones de usuarios, sino demostrar la capacidad de tomar decisiones técnicas justificadas, implementarlas de forma coherente dentro de un alcance definido y documentar el resultado con precisión. Esto incluye reconocer las limitaciones del sistema y distinguir con claridad qué está hecho, qué queda fuera del alcance y por qué.

1.3 Alcance del sistema

NutriFit cubre catorce áreas funcionales en su versión actual:

- **Autenticación:** registro, login y logout con invalidación inmediata del token en el servidor.
- **Catálogo de alimentos:** creación, consulta, búsqueda por nombre, actualización y eliminación, con valores nutricionales por 100 gramos.
- **Escáner de códigos de barras:** búsqueda de alimentos por código EAN en Open Food Facts.
- **Registro de comidas:** comidas asociadas a una fecha y tipo de toma, con adición y eliminación de ítems con gramos.
- **Resumen diario:** total de kilocalorías, proteínas, grasas e hidratos de carbono consumidos en una fecha.
- **Evaluación nutricional con IA:** análisis del resumen diario mediante un modelo de lenguaje (OpenRouter).
- **Perfil biométrico:** sexo, fecha de nacimiento, altura, peso y nivel de actividad; cálculo automático de TMB y TDEE.
- **Registro de ejercicios:** sesiones de actividad física con cálculo de calorías quemadas mediante factor MET.
- **Hidratación:** registro diario de vasos de agua consumidos.
- **Historial de peso:** seguimiento del peso corporal a lo largo del tiempo.
- **Plan semanal:** generación de un plan de comidas personalizado mediante IA.
- **Retos:** desafíos de fitness con seguimiento de progreso.
- **Lista de la compra:** generación y gestión de listas de compra con sugerencias de IA.
- **Gamificación:** racha de días activos y puntuación acumulada.

1.4 Arquitectura y tecnologías

El sistema se organiza en tres capas independientes. La capa de presentación tiene dos implementaciones: una aplicación web React desplegada en Vercel y un cliente de escritorio JavaFX. Ambas consumen el mismo backend mediante peticiones HTTP. El backend, implementado con Spring Boot 3.5, expone una API REST con dieciocho prefijos de endpoint y concentra toda la lógica de negocio y el acceso a datos. PostgreSQL almacena el estado del sistema bajo un esquema gestionado íntegramente por Flyway (veintidós migraciones).

El acceso a datos se realiza con JdbcTemplate y RowMapper manuales, sin ORM. La autenticación usa tokens opacos UUID v4 almacenados en base de datos. Los errores están centralizados en un `@RestControllerAdvice` que garantiza una estructura de respuesta uniforme para cualquier excepción.

1.5 Organización de la memoria

La memoria sigue el desarrollo del proyecto de forma progresiva:

- §2 — Estado del arte: revisión de soluciones existentes y marco teórico nutricional.
- §3 — Tecnologías y herramientas: stack completo con justificación de cada elección.

- §4 — Análisis de requisitos: requisitos funcionales y no funcionales.
- §5 — Diseño: arquitectura, esquema de base de datos y decisiones de diseño.
- §6 — Implementación: descripción módulo a módulo.
- §7 — Pruebas: estrategia y suite de 155 tests.
- §8 — Seguridad: medidas implementadas y limitaciones conocidas.
- §9 — Conclusiones y líneas futuras.

2. Estado del arte y marco teórico

2.1 El problema del seguimiento nutricional

El control de la ingesta calórica y la distribución de macronutrientes tiene interés en el ámbito clínico, deportivo y de la salud general. El registro sistemático de la alimentación es uno de los predictores más consistentes de éxito en programas de control de peso, según estudios publicados en revistas de nutrición y medicina preventiva.

El problema técnico es siempre el mismo: un sistema que relacione alimentos con sus valores nutricionales, que permita al usuario registrar qué cantidades ha consumido en qué momento del día, y que agregue esa información para ofrecer una visión del conjunto. Sobre ese núcleo básico se construyen los sistemas más complejos.

2.2 Soluciones comerciales existentes

2.2.1 MyFitnessPal

MyFitnessPal es la aplicación de seguimiento nutricional más utilizada a nivel mundial. Tiene una base de datos de más de 14 millones de alimentos, seguimiento de macros y micros, integración con dispositivos wearables y comunidad activa. Su limitación desde el punto de vista técnico es que opera como una caja negra: el usuario no puede conocer cómo se modelan las comidas internamente ni qué decisiones de diseño subyacen a su API.

2.2.2 Cronometer

Cronometer se diferencia de MyFitnessPal por su énfasis en la precisión nutricional, incluyendo vitaminas, minerales y aminoácidos. Usa bases de datos validadas científicamente. Su interfaz es más densa que la de MyFitnessPal, orientada a usuarios con conocimientos nutricionales previos.

2.2.3 Yazio

Yazio es una aplicación de origen alemán con buena acogida en Europa. Ofrece seguimiento de macros, planes de dieta personalizados y registro de ejercicio. Integra cálculo de TMB y TDEE de forma automática. Su limitación es la dependencia de su propia base de datos de alimentos, que en algunos mercados es menos completa.

2.3 Limitaciones formativas de las soluciones existentes

Estas aplicaciones resuelven el problema del usuario final pero no son útiles como referencia de aprendizaje técnico porque su código fuente no es accesible, no documentan sus decisiones de modelado y operan a una escala no reproducible en un entorno formativo.

NutriFit aborda el mismo problema funcional con un objetivo diferente: un sistema comprensible, documentado y reproducible, donde cada decisión técnica está justificada y es trazable hasta el código concreto que la implementa.

2.4 Marco teórico: fórmulas y conceptos nutricionales

2.4.1 Fórmula de Mifflin-St Jeor

El cálculo del gasto energético en NutriFit se basa en la fórmula de Mifflin-St Jeor, publicada en 1990 y considerada una de las más precisas para estimar la Tasa Metabólica Basal (TMB) en adultos sanos:

```
Hombres: TMB = (10 × peso_kg) + (6,25 × altura_cm) - (5 × edad) + 5  
Mujeres: TMB = (10 × peso_kg) + (6,25 × altura_cm) - (5 × edad) - 161
```

El Gasto Energético Total Diario (TDEE) se obtiene multiplicando la TMB por el factor de actividad correspondiente: 1,2 para sedentario, 1,375 para actividad ligera, 1,55 para moderada, 1,725 para alta y 1,9 para muy alta.

2.4.2 Factor MET para calorías quemadas

El Equivalente Metabólico (MET) expresa el coste energético de una actividad en relación al metabolismo basal en reposo; un MET equivale a aproximadamente 1 kcal/(kg·h). La fórmula aplicada en NutriFit es:

```
kcal = MET × peso_kg × (duración_min / 60)
```

Los valores MET de referencia están tabulados en el Compendium of Physical Activities de Ainsworth et al. NutriFit incluye en su catálogo los valores MET de las actividades aeróbicas y de fuerza más habituales.

2.5 Integración de inteligencia artificial

La versión actual incorpora capacidades de IA a través de OpenRouter, un proxy de API que permite seleccionar entre distintos modelos de lenguaje sin acoplarse a un proveedor. Las funciones de IA en NutriFit son tres: evaluación del resumen nutricional diario, generación de un plan de comidas semanal personalizado y sugerencias para la lista de la compra. El modelo concreto y la clave de API son configurables por el usuario.

3. Tecnologías y herramientas

3.1 Visión de conjunto

NutriFit está compuesto por tres módulos independientes: **backend** (Spring Boot, Maven), **client** (JavaFX, Maven) y **frontend** (React, Vite). Cada módulo selecciona sus dependencias en función de su responsabilidad; los tres comparten únicamente el protocolo HTTP y el formato JSON.

Tecnología	Versión	Módulo	Papel en el proyecto
Java	21 (LTS)	Backend + cliente	Lenguaje base
Spring Boot	3.5.0	Backend	Marco de aplicación: IoC, servidor web embebido, validación
Spring JDBC (JdbcTemplate)	gestionada por BOM	Backend	Acceso a datos con SQL directo y RowMapper manuales
Spring Security Crypto	gestionada por BOM	Backend	Codificación de contraseñas con BCrypt
PostgreSQL	16+	Base de datos	Motor relacional
postgresql (driver)	gestionada por BOM	Backend	Driver JDBC para PostgreSQL
Flyway	gestionada por BOM	Backend	Migraciones de esquema versionadas (V1–V22)
springdoc-openapi	2.8.5	Backend	Swagger UI y especificación OpenAPI 3
Apache Maven	3.8+	Backend + cliente	Construcción y gestión de dependencias
JavaFX	21	Cliente	Interfaz gráfica de escritorio
java.net.http.HttpClient	nativo Java 21	Cliente	Comunicación HTTP con el backend
Jackson Databind	2.17+	Cliente	Serialización y deserialización JSON
React	18	Frontend	Framework de interfaz web basado en componentes
TypeScript	5	Frontend	Tipado estático sobre JavaScript
Vite	6	Frontend	Bundler y servidor de desarrollo
Tailwind CSS	3	Frontend	Framework CSS utilitario
Framer Motion	11	Frontend	Animaciones declarativas en React

Vitest	3	Frontend	Marco de tests unitarios para Vite
JUnit 5	gestionada por BOM	Backend	Marco de tests unitarios
Mockito	gestionada por BOM	Backend	Mocks en tests unitarios
AssertJ	gestionada por BOM	Backend	Aserciones fluidas
OpenRouter API	—	Backend	Proxy de LLM para funciones de IA
Open Food Facts API	—	Backend	Base de datos de alimentos por código de barras
Git + GitHub	—	Repositorio	Control de versiones
GitHub Actions	—	CI/CD	Compilación automática en cada push
Docker + docker-compose	—	Infraestructura	Backend + PostgreSQL contenerizados (desarrollo local)
Render	—	Despliegue	Hosting del backend (plan gratuito)
Vercel	—	Despliegue	Hosting del frontend (plan gratuito)

3.2 Backend

3.2.1 Java 21

El proyecto compila con Java 21, la versión LTS más reciente. Java 21 aporta bloques de texto para las consultas SQL multilínea en los repositorios, la API `java.time` para el manejo de fechas en el dominio y mejoras en la gestión de hilos que, aunque no se utilizan directamente aquí, hacen que Java 21 sea la base más sólida disponible.

3.2.2 Spring Boot 3.5.0

Spring Boot gestiona el ciclo de vida del servidor, la inyección de dependencias, la validación de peticiones con Bean Validation y las conexiones JDBC. Se usan tres starters: `spring-boot-starter-web` para el servidor REST, `spring-boot-starter-validation` para validación declarativa y `spring-boot-starter-jdbc` para `JdbcTemplate` con pool HikariCP.

3.2.3 Spring JDBC y JdbcTemplate

El acceso a datos se implementa íntegramente con `JdbcTemplate` y `RowMapper` manuales. No se usa JPA, Spring Data ni Hibernate. Cada módulo declara una interfaz `xxxRepository` y una implementación `JdbcxxxRepository` con SQL explícito. La justificación de esta elección frente a JPA se desarrolla en §5.3.1.

3.2.4 PostgreSQL y Flyway

PostgreSQL 16 almacena el estado del sistema. El esquema no se define manualmente: Flyway aplica automáticamente las veintidós migraciones al arrancar el backend. Si el esquema ya está al día, Flyway lo valida y no ejecuta nada, garantizando que cualquier entorno arranca con el mismo esquema exacto.

3.2.5 Integración con APIs externas

OpenRouter actúa como proxy entre el backend y los modelos de lenguaje. El backend envía el contexto nutricional del usuario (resumen diario, perfil, historial) a OpenRouter y recibe la respuesta del modelo. La clave de API y el modelo concreto se almacenan por usuario en la tabla `usuario_ia_config`. Open Food Facts proporciona datos nutricionales por código de barras EAN; el backend realiza una llamada HTTP a su API pública y devuelve el alimento al cliente.

3.3 Frontend React

3.3.1 React 18 + TypeScript

La interfaz web es una SPA (Single Page Application) construida con React 18 y TypeScript. React gestiona el estado de la UI mediante hooks; TypeScript añade tipado estático que elimina una categoría entera de errores en tiempo de compilación. La interfaz tiene catorce páginas: Dashboard, Alimentos, Comidas, Ejercicios, Escáner, Hidratación, Lista de compra, Opciones IA, Perfil, Plan semanal, Retos, Tendencias, Login y Registro.

3.3.2 Vite y Tailwind CSS

Vite actúa como bundler y servidor de desarrollo. El tiempo de arranque en caliente es inferior a un segundo en la máquina de desarrollo. Tailwind CSS gestiona los estilos mediante clases utilitarias aplicadas directamente en JSX, sin archivos CSS separados por componente. Framer Motion añade transiciones declarativas entre páginas y en la aparición de elementos.

3.4 Cliente JavaFX

El cliente JavaFX mantiene las funciones del MVP original: acceso (login/registro), gestión de alimentos, diario nutricional y perfil de usuario. Consume el mismo backend mediante `java.net.http.HttpClient` y Jackson Databind. Las operaciones de red se ejecutan en hilos de fondo con `javafx.concurrent.Task`.

3.5 Despliegue

El backend se despliega en Render con build automático desde la rama `main`. El frontend se despliega en Vercel con el mismo trigger. Ambos servicios proporcionan HTTPS sin configuración adicional. El backend en el plan gratuito de Render se suspende tras 15 minutos de inactividad; la primera petición tras ese período tiene una latencia de arranque de 30-60 segundos.

4. Análisis de requisitos

4.1 Introducción

Los requisitos emergieron de forma iterativa a medida que los módulos se definían e implementaban. El objetivo de documentarlos aquí es hacer explícito qué hace el sistema, qué restricciones de calidad cumple y qué queda fuera del alcance actual.

4.2 Requisitos funcionales

Módulo de autenticación

Código	Requisito
RF-01	El sistema permite registrar un nuevo usuario con nombre, email y contraseña. El registro abre sesión de forma automática.
RF-02	Un usuario registrado puede autenticarse con su email y contraseña. Si las credenciales son correctas, el sistema genera un token de sesión.
RF-03	Un usuario puede cerrar su sesión. La invalidación del token es inmediata y del lado del servidor.
RF-04	El sistema normaliza el email antes de cualquier búsqueda: aplica <code>trim()</code> y <code>toLowerCase()</code> .
RF-05	No pueden existir dos usuarios con el mismo email. El intento de registro con email duplicado se rechaza antes de persistir ningún dato.

Módulo de alimentos

Código	Requisito
RF-06	El sistema mantiene un catálogo de alimentos compartido. Cualquier usuario puede consultar el catálogo completo o filtrar por nombre.
RF-07	La búsqueda por nombre es parcial e insensible a mayúsculas.
RF-08	Se puede crear un alimento especificando nombre, porción de referencia, kcal/100g, proteínas/100g, grasas/100g e hidratos/100g.
RF-09	Un alimento existente puede actualizarse o eliminarse. No se puede eliminar un alimento referenciado en algún ítem de comida.
RF-10	El escáner busca un alimento en Open Food Facts por código de barras EAN y devuelve sus datos nutricionales para añadirlo al catálogo.

Módulo de comidas

Código	Requisito
RF-11	Un usuario puede registrar una comida para una fecha concreta con un tipo de toma (DESAYUNO, ALMUERZO, CENA, etc.).
RF-12	A una comida se le pueden añadir ítems. Cada ítem asocia un alimento del catálogo con una cantidad en gramos.
RF-13	Al consultar los ítems de una comida, el sistema calcula y devuelve los valores nutricionales ponderados por gramos. El cálculo se realiza en base de datos.
RF-14	Una comida puede eliminarse. Al hacerlo, todos sus ítems se eliminan en cascada.

Módulo de resumen diario y evaluación IA

Código	Requisito
RF-15	El sistema calcula y devuelve el total de kcal, proteínas, grasas e hidratos consumidos por un usuario en una fecha.
RF-16	Si el usuario no tiene comidas en la fecha consultada, el resumen devuelve ceros. No se produce error.
RF-17	El sistema genera una evaluación nutricional del resumen diario mediante un modelo de lenguaje. El resultado incluye análisis de macros, comparativa con el TDEE y recomendaciones.

Módulo de perfil

Código	Requisito
RF-18	Un usuario puede configurar su perfil biométrico: sexo, fecha de nacimiento, altura, peso actual, nivel de actividad y, opcionalmente, peso objetivo.
RF-19	El sistema calcula la TMB y el TDEE a partir del perfil, usando la fórmula de Mifflin-St Jeor. El cálculo se realiza en el backend.
RF-20	Un usuario puede actualizar su perfil en cualquier momento. El sistema recalcula TMB y TDEE de forma automática.

Módulo de ejercicios

Código	Requisito
RF-21	El sistema mantiene un catálogo de ejercicios con su valor MET. Cualquier usuario autenticado puede consultarlo.
RF-22	Un usuario puede registrar una sesión de ejercicio indicando el tipo, la fecha y la duración en minutos.
RF-23	El sistema calcula las calorías quemadas: $MET \times peso_kg \times (duración_min / 60)$, redondeando a dos decimales.
RF-24	El peso utilizado en el cálculo se toma del perfil en el momento del registro y se persiste junto al resultado.

Módulos adicionales

Código	Módulo	Requisito
RF-25	Hidratación	El usuario puede registrar vasos de agua consumidos por día. El sistema devuelve el total del día.
RF-26	Peso historial	El usuario puede registrar su peso en distintas fechas. El sistema devuelve el historial ordenado cronológicamente.
RF-27	Tendencias	El sistema calcula tendencias nutricionales de los últimos 90 días: medias de kcal, macros e hidratación.
RF-28	Plan semanal	El sistema genera un plan de comidas para la semana mediante IA, adaptado al perfil y objetivos del usuario.
RF-29	Lista de compra	El usuario puede gestionar una lista de la compra. El sistema puede generar sugerencias mediante IA basadas en el historial de comidas.
RF-30	Retos	El sistema mantiene un catálogo de retos de fitness. El usuario puede inscribirse y registrar progreso.
RF-31	Gamificación	El sistema calcula una racha de días activos y una puntuación acumulada basada en el uso de las funciones del sistema.
RF-32	Config. IA	El usuario puede configurar el modelo de IA y la clave de API de OpenRouter que se usará en sus evaluaciones.

4.3 Requisitos no funcionales

Código	Requisito
RNF-01	Todos los campos obligatorios se validan con Bean Validation antes de que la petición llegue al servicio. Restricciones incumplidas devuelven HTTP 400.
RNF-02	Los valores numéricos de alimentos no pueden ser negativos. La altura del perfil debe estar entre 100 y 250 cm; el peso, por encima de 20 kg.
RNF-03	Todos los errores del backend tienen la misma estructura JSON: <code>timestamp</code> , <code>status</code> , <code>error</code> , <code>message</code> y <code>path</code> .
RNF-04	Los mensajes de error de login son idénticos para email no encontrado y contraseña incorrecta, para impedir enumeración de usuarios.
RNF-05	El esquema de la base de datos está gestionado exclusivamente por Flyway. El código Java nunca crea ni modifica tablas.
RNF-06	Las contraseñas se almacenan exclusivamente como hash BCrypt. El texto en claro nunca se persiste ni se registra.
RNF-07	Los tokens de sesión son UUIDs v4 generados con <code>SecureRandom</code> , almacenados con expiración de siete días. Al hacer logout, la fila se elimina.

RNF-08	Todas las operaciones de red del cliente JavaFX se ejecutan en hilos de fondo con <code>javafx.concurrent.Task</code> .
RNF-09	Los tests unitarios del backend se ejecutan sin base de datos activa ni contexto de Spring. Los repositorios son mocks de Mockito.
RNF-10	El frontend y el backend están desplegados en HTTPS. La comunicación entre cliente web y servidor nunca se produce en texto plano.
RNF-11	El backend acepta peticiones únicamente desde los orígenes configurados en CORS (<code>https://nutri-fit-snowy.vercel.app</code> y <code>http://localhost:5173</code>).

4.4 Exclusiones conocidas

El sistema actual no implementa diferenciación de roles entre usuarios, no tiene rate limiting en las llamadas a OpenRouter, y el catálogo de alimentos es compartido sin control de acceso por usuario. Estas limitaciones son consecuencia del alcance del MVP, no de errores de diseño.

5. Diseño del sistema

5.1 Arquitectura

NutriFit tiene tres capas con separación física:

Capa	Tecnología	Responsabilidad principal
Presentación web	React 18 + TypeScript (Vercel)	SPA, enrutado client-side, llamadas a la API
Presentación escritorio	JavaFX 21	Interfaz gráfica de escritorio, llamadas HTTP
Lógica de aplicación	Spring Boot 3.5 (Render, puerto 8080/443)	API REST, reglas de negocio, acceso a datos
Persistencia	PostgreSQL 16 (base de datos <code>nutrifit</code>)	Almacenamiento relacional

El backend no sabe nada del cliente; cualquier consumidor que respete el contrato HTTP obtiene el mismo comportamiento. Ambos clientes son módulos independientes que pueden construirse y probarse por separado.

5.1.1 Capa de presentación: frontend React

El frontend es una SPA servida desde Vercel. La navegación es client-side mediante React Router; las páginas no se recargan desde el servidor. El enrutado protegido se implementa con un componente `ProtectedRoute` que verifica el token de sesión en el contexto de autenticación antes de renderizar la página. Las peticiones a la API se centralizan en funciones del directorio `src/api/`, una por módulo.

5.1.2 Capa de presentación: cliente JavaFX

El cliente JavaFX cubre las funciones del MVP original. Dentro del cliente, los componentes se organizan en tres grupos: pantallas FXML con sus controladores, clases `ApiClient` por módulo (que encapsulan las llamadas HTTP), y `SessionManager` (clase estática que mantiene en memoria el token y los datos de sesión). El token no se persiste en disco; desaparece al cerrar la aplicación.

5.1.3 Capa de lógica de aplicación: el backend Spring Boot

El backend sigue la misma estructura en capas verticales para todos sus módulos:

```
Controller → Service → Repository → JdbcTemplate → PostgreSQL
```

Los controladores reciben la petición HTTP, validan con `@Valid` y delegan en el servicio. Los servicios contienen las reglas de negocio. Los repositorios encapsulan el acceso JDBC. Un

`GlobalExceptionHandler` centralizado convierte cualquier excepción en una respuesta JSON con estructura uniforme.

5.1.4 Capa de persistencia: PostgreSQL y Flyway

PostgreSQL almacena el estado del sistema. El esquema se construye exclusivamente mediante las veintidós migraciones de Flyway, almacenadas en `backend/src/main/resources/db/migration/`. Si el esquema ya está al día cuando el backend arranca, Flyway lo valida y no ejecuta nada.

5.2 Diseño de la base de datos

5.2.1 Migraciones Flyway

Migración	Contenido
V1, V2	Tablas <code>usuarios</code> y <code>alimentos</code>
V3	Tabla <code>sesiones</code>
V4	Seed inicial del catálogo de alimentos
V5	Tablas <code>comidas</code> y <code>comida_alimentos</code>
V6, V7	Tabla <code>ejercicios</code> y seed de tipos de ejercicio
V8, V9	Stored procedure del resumen diario y actualización
V10	Índices de rendimiento
V11	Seed expandido de alimentos
V12	Tabla <code>peso_historial</code>
V13	Índices adicionales y restricciones
V14	Tabla <code>hidratacion</code>
V15	Tabla <code>plan_semanal</code>
V16	Tabla <code>retos</code> y <code>usuario_retos</code>
V17	Tabla <code>lista_compra</code>
V18	Tabla <code>ejercicios_tipo</code>
V19	Seed completo del catálogo de ejercicios
V20	Campos de intensidad y modo anaeróbico en ejercicios
V21	Relajación de restricción de duración mínima
V22	Tabla <code>usuario_ia_config</code>

5.2.2 Principales relaciones

Entidad origen	Cardinalidad	Entidad destino	Al borrar origen
usuarios	1 — 0..*	sesiones	ON DELETE CASCADE
usuarios	1 — 0..*	comidas, peso_historial, hidratacion, plan_semanal, usuario_retos, lista_compra, usuario_ia_config	ON DELETE CASCADE
comidas	1 — 0..*	comida_alimentos	ON DELETE CASCADE
alimentos	1 — 0..*	comida_alimentos	ON DELETE RESTRICT
ejercicios	1 — 0..*	ejercicios_registro	ON DELETE RESTRICT

La regla `ON DELETE RESTRICT` en alimentos y ejercicios impide eliminar un elemento del catálogo si existe algún ítem que lo referencia, protegiendo la integridad del historial.

5.3 Decisiones de diseño

5.3.1 Acceso a datos con Spring JDBC en lugar de JPA/Hibernate

El acceso a la base de datos se implementa con `JdbcTemplate` y `RowMapper` manuales. No se usa JPA ni Hibernate. El argumento principal es de trazabilidad: con `JdbcTemplate`, cada consulta es SQL visible en una línea concreta del código. En una defensa, responder "¿qué SQL genera esta línea?" es inmediato con JDBC y requiere depuración con JPA. Como ventaja adicional, Flyway gestiona el esquema sin ninguna interferencia; no hay que coordinar `ddl-auto`.

5.3.2 Autenticación con token opaco en lugar de JWT

La autenticación usa un UUID aleatorio almacenado en la tabla `sesiones`, con fecha de expiración y borrado explícito al hacer logout. JWT no resuelve bien el logout: un token JWT firmado sigue siendo válido hasta que expira, aunque el usuario lo haya invalidado. Forzar la invalidación inmediata con JWT requeriría una lista negra en base de datos, que es exactamente la misma infraestructura que ya se tiene, pero con mayor complejidad.

5.3.3 Cálculo del resumen diario con SQL

El resumen diario se calcula mediante una única consulta SQL con `LEFT JOIN`, `SUM`, `ROUND` y `COALESCE`. La alternativa descartada era traer todas las comidas e ítems al servicio Java y calcular los totales en memoria. Eso habría requerido múltiples consultas y lógica de agregación en la capa de aplicación, desaprovechando las capacidades del motor de base de datos.

5.3.4 Cálculo de TMB/TDEE en el backend

La fórmula de Mifflin-St Jeor reside en el servicio de backend. Los valores calculados se devuelven en la respuesta. Esto tiene dos consecuencias prácticas: la lógica está en un único lugar y puede verificarse con tests unitarios independientes del cliente; y si el cliente cambia de tecnología, la fórmula no se duplica.

5.3.5 Separación de la configuración de IA por usuario

La clave de API de OpenRouter y el modelo concreto se almacenan por usuario en `usuario_ia_config`, no en variables de entorno globales. Esto permite que cada usuario configure su propia integración sin que el administrador del sistema tenga que gestionar una clave compartida. El inconveniente es que el usuario necesita tener una cuenta en OpenRouter, lo que añade fricción en el onboarding.

5.3.6 PostgreSQL en lugar de MariaDB

El proyecto evolucionó de MariaDB (borrador anterior) a PostgreSQL. Los motivos son prácticos: Render, el proveedor de hosting elegido, ofrece PostgreSQL gestionado en su plan gratuito pero no MariaDB. Además, PostgreSQL tiene un soporte más completo para tipos de datos JSON, que se usa en las respuestas de la API de IA.

6. Implementación

6.1 Módulo de autenticación

El módulo expone tres endpoints bajo `/api/auth`: `POST /register` (201), `POST /login` (200) y `POST /logout` (204). El flujo de registro aplica `trim()` y `toLowerCase()` al email, comprueba unicidad, hashlea la contraseña con BCrypt, persiste el usuario, genera un UUID v4 como token de sesión y devuelve el `AuthResponse` con los datos del usuario y el token. El registro abre sesión automáticamente.

En el login, si el email no existe o la contraseña no coincide, el servicio lanza la misma excepción con el mismo mensaje: *"Credenciales inválidas"*. Esto impide que un atacante deduzca si un email está registrado.

6.2 Módulo de alimentos y escáner

El módulo mantiene un catálogo compartido con CRUD completo. Los valores nutricionales se modelan como `BigDecimal` para preservar la precisión decimal. La búsqueda por nombre usa LIKE con comodines, insensible a mayúsculas gracias a la collation de la base de datos.

El escáner se implementa en `GET /api/escaner/{barcode}`. El backend realiza una llamada a la API pública de Open Food Facts, extrae los campos nutricionales del producto y los devuelve en el mismo formato que un alimento del catálogo. Si el código no existe en Open Food Facts, el endpoint devuelve 404.

6.3 Módulo de comidas

El módulo distingue dos entidades: `Comida` (una toma del día, identificada por usuario, fecha y tipo) y `ComidaAlimento` (la relación entre una comida y un alimento del catálogo, con los gramos consumidos). Al añadir un ítem, el servicio verifica primero que tanto la comida como el alimento existen, aplicando el patrón fail-fast.

6.4 Módulo de resumen diario y evaluación IA

El resumen diario se calcula mediante una única consulta SQL que cruza tres tablas (`comidas`, `comida_alimentos`, `alimentos`) con dos LEFT JOIN. La fórmula por ítem es $(\text{valor_por_100g} \times \text{gramos}) / 100$. COALESCE convierte los nulos que produce SUM sobre un conjunto vacío en ceros.

La evaluación IA toma el resumen del día, el perfil biométrico del usuario y su historial reciente, construye un prompt estructurado y lo envía a OpenRouter. La respuesta del modelo, en texto libre, se devuelve al cliente tal cual. Si el usuario no tiene configurada una clave de API, el endpoint devuelve 400 con un mensaje que explica el requisito.

6.5 Módulo de perfil y cálculo TMB/TDEE

Los datos biométricos se almacenan directamente en la tabla `usuarios`. El módulo expone dos endpoints: `GET /api/perfil/{id}` y `PUT /api/perfil/{id}`. La fórmula de Mifflin-St Jeor se implementa en `PerfilServiceImpl`:

```
private double calcularTmb(Perfil perfil) {
    int edad = Period.between(perfil.getFechaNacimiento(),
        LocalDate.now()).getYears();
    double base = 10 * perfil.getPesoKgActual()
        + 6.25 * perfil.getAlturaCm()
        - 5 * edad;
    return perfil.getSexo() == Sexo.H ? base + 5 : base - 161;
}
```

La edad se calcula con `Period.between` para garantizar el resultado correcto independientemente del día de ejecución.

6.6 Módulo de ejercicios

El módulo gestiona el catálogo de tipos de ejercicio y los registros de sesiones. Las calorías quemadas se calculan con $MET \times peso_kg \times (duración_min / 60.0)$, redondeando a dos decimales con `HALF_UP`. El peso se toma del perfil en el momento del registro y se persiste junto al resultado para que el historial no cambie si el usuario actualiza su peso posteriormente.

6.7 Módulos adicionales

Los módulos de hidratación, peso historial, retos, plan semanal y lista de compra siguen la misma estructura `Controller` → `Service` → `Repository`. Los módulos con IA (plan semanal, lista de compra, evaluación) delegan en `OpenRouterService`, que construye el prompt, gestiona la llamada HTTP a OpenRouter y propaga los errores de cuota o autenticación como excepciones controladas. El módulo de gamificación calcula la racha de días activos consultando los registros de comidas, ejercicios e hidratación, y la puntuación acumulada sumando actividades del último mes.

6.8 Frontend React

El frontend organiza las llamadas a la API en funciones del directorio `src/api/`. Cada función construye la petición con `fetch`, añade el token de sesión en la cabecera `Authorization`, verifica el código de estado y lanza un error si no es 2xx. El estado de autenticación se gestiona mediante un contexto React (`AuthContext`) que expone el usuario actual, el token y las funciones de login y logout.

Los componentes de visualización incluyen gráficos de tendencias de peso (`WeightChart`), distribución de macronutrientes (`MacrosChart`, `MacroRing`), heatmap de ejercicios (`EjercicioHeatmap`) y puntuación nutricional (`NutriScoreChart`).

6.9 Cliente JavaFX

El cliente organiza la comunicación con el backend en clases `ApiClient` por módulo. Cada `ApiClient` construye la petición con `HttpRequest.newBuilder()`, serializa el cuerpo con Jackson si aplica, envía la petición y verifica el código de estado. Si no es 2xx, lanza una excepción con el mensaje del cuerpo de error. Todas las operaciones de red se envuelven en `javafx.concurrent.Task`.

6.10 Integración continua y despliegue

GitHub Actions compila el backend y el cliente JavaFX con Java 21 en cada push, garantizando que ningún commit deja el proyecto en estado no compilable. El frontend se despliega automáticamente en Vercel en cada push a `main`; el backend en Render con el mismo trigger. Las migraciones de Flyway se aplican automáticamente en el primer arranque del contenedor o al desplegar en Render.

7. Pruebas

7.1 Estrategia de pruebas

La estrategia combina tres enfoques: pruebas unitarias automatizadas sobre la capa de servicio del backend, pruebas unitarias sobre el cliente JavaFX y pruebas del frontend con Vitest. Los repositorios y controladores se verifican mediante pruebas manuales con peticiones HTTP reales contra el servidor en ejecución.

La cobertura automatizada se centra en la capa de servicio porque es ahí donde reside la lógica de negocio: validación de reglas de dominio, manejo de casos de error, transformación de datos y cálculos nutricionales. Los repositorios son envolturas directas sobre SQL y su comportamiento queda cubierto por las pruebas manuales de la API.

7.2 Suite de tests del backend

Las pruebas del backend usan JUnit 5 como marco de ejecución, Mockito para sustituir los colaboradores por objetos simulados, y AssertJ para las aserciones. Ningún test levanta el contexto de Spring: todos usan `@ExtendWith(MockitoExtension.class)`, lo que permite ejecutar la suite completa en menos de tres segundos sin ningún servicio externo activo.

Clase de test	Tests	Operaciones cubiertas
AlimentoServiceImplTest	12	findAll, findById, save, update, deleteById
AuthServiceImplTest	9	register, login, logout
ComidaServiceImplTest	17	save, findByUsuarioAndFecha, deleteById, addAlimentoToComida, findDetalleItemsByComidaId, deleteItem
ResumenDiarioServiceImplTest	3	obtenerResumenDiario
PerfilServiceImplTest	5	getPerfil, updatePerfil
RegistroEjercicioServiceImplTest	14	registrar, findByUsuarioAndFecha, deleteById, calcularKcal
HidratacionServiceImplTest	8	registrar, obtenerDia, borrar
PesoHistorialServiceImplTest	7	registrar, obtenerHistorial, borrar
GamificacionServiceTest	10	calcularRacha, obtenerPuntuacion
BaseIntegrationTest	6	Infraestructura de tests de integración
Total backend	114	

7.3 Suite de tests del cliente JavaFX

Clase de test	Tests	Operaciones cubiertas
AlimentoFxTest	8	Serialización y deserialización de DTOs de alimentos
BaseApiClientTest	5	Comportamiento base del cliente HTTP
PerfilDtoTest	4	Deserialización del DTO de perfil con TMB y TDEE
ResumenDiarioDtoTest	4	Deserialización del resumen diario con ceros y valores reales
SessionManagerTest	3	Gestión de sesión en memoria: set, get, clear
Total JavaFX	24	

7.4 Tests del frontend

El frontend tiene tres tests con Vitest que cubren los componentes más críticos de la UI. Por el alcance del proyecto, la mayor parte de la verificación del frontend se hace mediante pruebas manuales en el navegador y en el entorno de producción desplegado en Vercel.

Módulo	Tests	Framework
Backend	114	JUnit 5 + Mockito + AssertJ
Cliente JavaFX	24	JUnit 5 + Mockito + AssertJ
Frontend React	3	Vitest
Total	118	

7.5 Aspectos destacados de la suite

Los tests de `AuthServiceImplTest` verifican que el registro con email duplicado lanza `BadRequestException` antes de invocar `save` en ningún repositorio; que la sesión creada tiene el token correcto y expiración futura, comprobado con `ArgumentCaptor<Sesion>`; y que los mensajes de error son idénticos para email no encontrado y contraseña incorrecta.

Los tests de `PerfilServiceImplTest` verifican la fórmula de Mifflin-St Jeor con valores calculados analíticamente para cada sexo. Para evitar dependencia de la fecha de ejecución, la fecha de nacimiento se construye como `LocalDate.now().minusYears(30)`.

Los tests de `RegistroEjercicioServiceImplTest` verifican la fórmula MET con cuatro casos, incluyendo el redondeo: MET=5.0, 70 kg, 20 min produce 116,67 kcal (no 116,66). El resultado de la suite completa del backend es: **Tests run: 114, Failures: 0, Errors: 0, Skipped: 0.**

7.6 Pruebas manuales de la API

Todos los endpoints se han verificado con peticiones HTTP reales. El directorio `docs/api/` contiene archivos `.http` compatibles con IntelliJ IDEA y con la extensión REST Client de VS Code, organizados por módulo. Swagger UI, accesible en `http://localhost:8080/swagger-ui.html`, complementa las pruebas permitiendo explorar y ejecutar peticiones directamente desde el navegador.

8. Seguridad

8.1 Gestión de contraseñas

Las contraseñas nunca se almacenan en texto plano. Al registrarse, `AuthServiceImpl` delega el hash en `PasswordService`, que usa `BCryptPasswordEncoder` de Spring Security. BCrypt aplica una función de derivación de clave con sal aleatoria incorporada, de modo que dos hashes del mismo texto producen valores distintos, protegiéndose frente a ataques de diccionario precomputados.

Durante el login, `matches()` compara la contraseña recibida contra el hash almacenado. Si la verificación falla, el servicio lanza la misma excepción con el mismo mensaje que cuando el email no existe. Esta propiedad está verificada por dos tests unitarios en `AuthServiceImplTest`.

8.2 Autenticación basada en token opaco

NutriFit usa un token opaco, un UUID aleatorio, almacenado en la tabla `sesiones` de PostgreSQL. El token se genera con `UUID.randomUUID()`, que internamente usa `SecureRandom`. La columna `token` tiene restricción UNIQUE, impidiendo colisiones en base de datos.

En cada petición autenticada, `AuthInterceptor` extrae el token de la cabecera `Authorization`, consulta `SesionRepository.findByToken()` y verifica que la sesión existe y que `expires_at` es posterior al momento actual. Si alguna comprobación falla, lanza `UnauthorizedException`, que `GlobalExceptionHandler` convierte en HTTP 401.

Al hacer logout, el servicio elimina la fila correspondiente de `sesiones`. Tras esa llamada, el token no existe y cualquier petición posterior recibe HTTP 401. El cliente JavaFX almacena el token exclusivamente en memoria; el frontend lo guarda en `localStorage` del navegador para persistir la sesión entre recargas.

8.3 Transporte seguro y CORS

El backend está desplegado en Render con HTTPS obligatorio; el frontend en Vercel, también con HTTPS. La comunicación entre el frontend web y la API nunca se produce en texto plano.

El backend acepta peticiones únicamente desde los orígenes configurados en `WebMvcConfig`: `https://nutri-fit-snowy.vercel.app` y `http://localhost:5173`. Cualquier otra procedencia recibe un error de CORS. El cliente JavaFX se comunica directamente con el backend sin pasar por el navegador, por lo que no está sujeto a la política de CORS.

8.4 Privacidad y datos de salud

NutriFit almacena datos de salud del usuario: perfil biométrico, historial de peso, registro de comidas y ejercicios, e hidratación. Las medidas implementadas son: hash BCrypt para contraseñas, eliminación en cascada de todos los datos del usuario al borrar la cuenta, y expiración de sesiones a los siete días. No se comparten datos con terceros salvo las llamadas a OpenRouter, que el usuario configura con su propia clave de API.

8.5 Limitaciones conocidas

El umbral mínimo de contraseña es de seis caracteres, que es bajo respecto a los estándares habituales de producción. Esta limitación se mantiene para simplificar las pruebas. No hay diferenciación de roles entre usuarios: todos los autenticados tienen el mismo nivel de acceso. El catálogo de alimentos y ejercicios es compartido sin control de acceso por usuario.

9. Conclusiones y líneas futuras

9.1 Valoración del proyecto

NutriFit es un sistema funcional y desplegado que permite registrar alimentos, comidas y ejercicios, calcular el gasto energético estimado, evaluar la dieta con inteligencia artificial y visualizar tendencias a lo largo del tiempo. Se sostiene sobre una arquitectura de cuatro capas (frontend React, cliente JavaFX, backend Spring Boot y PostgreSQL) en la que cada componente tiene responsabilidades delimitadas y se comunica con el siguiente a través de interfaces bien definidas.

El sistema no es un prototipo: los dieciocho prefijos de endpoint están implementados, probados y desplegados en producción accesible públicamente. El esquema está gestionado por Flyway con veintidós migraciones. Los 118 tests cubren la lógica de negocio relevante y pasan sin base de datos ni contexto de Spring activo.

9.2 Aportación del sistema

NutriFit demuestra que es posible construir un sistema multicapa completo dentro del alcance de un TFG, partiendo de decisiones técnicas justificadas. Las más relevantes son: acceso a datos con JdbcTemplate sin ORM, que mantiene las consultas SQL visibles y trazables; autenticación con token opaco en base de datos, que hace el logout inmediato y real; cálculo del resumen diario con una única consulta SQL de agregación; integración de modelos de lenguaje sin acoplarse a un proveedor concreto; y manejo de errores centralizado con estructura de respuesta uniforme.

La separación entre clientes y backend tiene también una consecuencia verificable: el backend puede arrancarse, probarse y depurarse de forma completamente independiente de cualquier interfaz. El cliente web y el cliente de escritorio consumen exactamente la misma API.

9.3 Aprendizajes técnicos

Gestión del esquema con migraciones versionadas. El uso de Flyway obliga a tratar el esquema como algo explícito y versionado, no como un efecto secundario del modelo de datos. Cada cambio requiere un script numerado; el estado de la base de datos en cualquier entorno es predecible y reproducible.

Tests unitarios sin infraestructura. Diseñar los servicios con interfaces de repositorio permite sustituirlos por mocks de Mockito en los tests. Los tests son el resultado de esa decisión de diseño, no su punto de partida.

Integración de LLMs en aplicaciones Java. Conectar un modelo de lenguaje no es complejo técnicamente, pero requiere gestionar casos de fallo que no existen en APIs más predecibles: timeouts, respuestas malformadas, claves expiradas y variabilidad en el formato de la respuesta.

Despliegue real frente a despliegue local. Los problemas que aparecen al desplegar (cold start de Render, CORS al cambiar la URL de Vercel, diferencias de collation entre PostgreSQL local y en producción) no son visibles en local. El despliegue continuo desde el primer mes del proyecto los expuso antes de que acumularan.

9.4 Limitaciones del sistema actual

Las limitaciones más relevantes son el catálogo de alimentos compartido sin control de acceso por usuario, la ausencia de rate limiting en las llamadas a OpenRouter, el umbral bajo de contraseña y la falta de diferenciación de roles. Estas limitaciones son consecuencia del alcance definido para el MVP, no de errores de diseño, y están identificadas explícitamente en la documentación.

9.5 Líneas futuras de trabajo

- Añadir control de acceso por usuario en el catálogo de alimentos, de modo que cada usuario gestione su propio subconjunto.
- Implementar rate limiting en los endpoints de IA para evitar un consumo excesivo de la clave de API del usuario.
- Completar la pantalla de comidas en el cliente JavaFX, que actualmente solo está cubierta a nivel de backend y frontend web.
- Ampliar los tests del frontend con Vitest hasta cubrir los componentes críticos de visualización.
- Añadir notificaciones push o recordatorios de hidratación y registro de comidas.
- Explorar la integración de la API de USDA FoodData Central como fuente de datos nutricionales adicional al catálogo de Open Food Facts.

9.6 Cierre

NutriFit es un sistema funcional dentro de su alcance definido. Sus decisiones de diseño están documentadas, sus limitaciones están identificadas y sus pruebas cubren la lógica de negocio del backend. No es un sistema terminado en todos sus aspectos, pero sí es uno del que se puede razonar con precisión: qué hace, qué no hace y por qué.

10. Bibliografía

Documentación oficial

Backend

- Spring Boot Reference Documentation (3.x). Broadcom / VMware.
<https://docs.spring.io/spring-boot/docs/current/reference/html/>
- Spring Framework — JdbcTemplate and JDBC Core Classes.
<https://docs.spring.io/spring-framework/reference/data-access/jdbc/core.html>
- Spring Security Crypto — Password Encoding.
<https://docs.spring.io/spring-security/reference/features/authentication/password-storage.html>
- Flyway Documentation — Migrations. Redgate.
<https://documentation.redgate.com/flyway>
- PostgreSQL 16 Documentation. PostgreSQL Global Development Group.
<https://www.postgresql.org/docs/16/>
- springdoc-openapi — OpenAPI 3 Library for Spring Boot.
<https://springdoc.org/>
- Apache Maven — Introduction to the POM. Apache Software Foundation.
<https://maven.apache.org/guides/introduction/introduction-to-the-pom.html>

Frontend

- React Documentation. Meta Open Source. <https://react.dev/>
- TypeScript Handbook. Microsoft. <https://www.typescriptlang.org/docs/>
- Vite Documentation. Evan You / VoidZero. <https://vitejs.dev/guide/>
- Tailwind CSS Documentation. Tailwind Labs. <https://tailwindcss.com/docs/>
- Framer Motion API Reference. Framer. <https://www.framer.com/motion/>
- Vitest Documentation. VoidZero. <https://vitest.dev/guide/>

Cliente JavaFX

- OpenJFX Documentation — JavaFX 21. <https://openjfx.io/openjfx-docs/>

Tests

- JUnit 5 User Guide. <https://junit.org/junit5/docs/current/user-guide/>
- Mockito Documentation. <https://site.mockito.org/>

- AssertJ — Fluent assertions for Java. <https://assertj.github.io/doc/>

APIs externas

- OpenRouter Documentation. OpenRouter. <https://openrouter.ai/docs>
 - Open Food Facts API Documentation. Open Food Facts.
<https://wiki.openfoodfacts.org/API>
-

Referencias nutricionales y fisiológicas

- Mifflin, M.D., St Jeor, S.T., Hill, L.A., Scott, B.J., Daugherty, S.A., Koh, Y.O. (1990). A new predictive equation for resting energy expenditure in healthy individuals. *American Journal of Clinical Nutrition*, 51(2), 241–247. <https://doi.org/10.1093/ajcn/51.2.241>
 - Ainsworth, B.E., Haskell, W.L., Herrmann, S.D., et al. (2011). 2011 Compendium of Physical Activities: A second update of codes and MET values. *Medicine & Science in Sports & Exercise*, 43(8), 1575–1581. <https://doi.org/10.1249/MSS.0b013e31821ece12>
-

Herramientas y estándares

- Leach, P., Mealling, M., Salz, R. (2005). RFC 4122 — A Universally Unique Identifier (UUID) URN Namespace. IETF. <https://www.rfc-editor.org/rfc/rfc4122>
 - Provos, N., Mazières, D. (1999). A Future-Adaptable Password Scheme. *USENIX Annual Technical Conference*, Freenix Track. <https://www.usenix.org/legacy/events/usenix99/provos.html>
 - GitHub Actions Documentation. GitHub. <https://docs.github.com/en/actions>
 - Docker Documentation. Docker, Inc. <https://docs.docker.com/>
 - Render Documentation. Render Services, Inc. <https://render.com/docs>
 - Vercel Documentation. Vercel, Inc. <https://vercel.com/docs>
-

Aplicaciones de referencia consultadas

- MyFitnessPal. Under Armour. <https://www.myfitnesspal.com/>
- Cronometer. Cronometer Software Inc. <https://cronometer.com/>
- Yazio. YAZIO GmbH. <https://www.yazio.com/>

Anexo A — Guía de puesta en marcha

A.1 Requisitos previos

Herramienta	Versión mínima	Uso
Java (JDK)	21	Backend y cliente JavaFX
Maven	3.8	Gestión de dependencias y arranque
PostgreSQL	14+	Base de datos del backend
Node.js	18+	Frontend React
npm	9+	Gestión de dependencias del frontend
Git	cualquiera	Clonación del repositorio

A.2 Preparación de la base de datos

Ejecutar en PostgreSQL como administrador:

```
CREATE DATABASE nutrifit;  
CREATE USER nutrifit WITH PASSWORD 'nutrifit123';  
GRANT ALL PRIVILEGES ON DATABASE nutrifit TO nutrifit;
```

No hace falta crear las tablas manualmente. Flyway las aplica al arrancar el backend.

A.3 Configuración del backend

Crear el archivo `backend/src/main/resources/application-local.properties`:

```
spring.datasource.url=jdbc:postgresql://localhost:5432/nutrifit  
spring.datasource.username=nutrifit  
spring.datasource.password=nutrifit123  
spring.datasource.driver-class-name=org.postgresql.Driver  
  
spring.flyway.enabled=true  
spring.flyway.locations=classpath:db/migration  
  
server.port=8080
```

A.4 Arranque del backend

```
cd backend  
mvn -DskipTests spring-boot:run "-Dspring-boot.run.profiles=local"
```


El backend está listo cuando el log muestra: `Tomcat started on port 8080 (http)`. Swagger UI disponible en `http://localhost:8080/swagger-ui.html`.

A.5 Arranque del frontend

Crear el archivo `frontend/.env.local`:

```
VITE_API_URL=http://localhost:8080
```

```
cd frontend
npm install
npm run dev
```

El frontend arranca en `http://localhost:5173`.

A.6 Arranque del cliente JavaFX

```
cd client
mvn javafx:run
```

El cliente debe arrancarse después del backend. La URL del backend por defecto es `http://localhost:8080`.

A.7 Orden de arranque

El orden correcto es: (1) arrancar PostgreSQL, (2) arrancar el backend, (3) arrancar el frontend o el cliente JavaFX. El frontend y el cliente JavaFX son independientes entre sí.

A.8 Opción Docker (desarrollo local)

```
docker-compose up --build
```

Este comando levanta el backend y PostgreSQL contenerizados. El healthcheck de PostgreSQL garantiza que el backend no intenta conectarse hasta que la base de datos está lista.

A.9 Ejecución de tests

```
# Tests del backend
cd backend && mvn test

# Tests del cliente JavaFX
cd client && mvn test
```

```
# Tests del frontend
cd frontend && npm test
```

A.10 Problemas frecuentes

El backend falla con `UnsupportedClassVersionError`. El proyecto requiere Java 21. Verificar la versión activa con `java -version`.

Error de conexión a la base de datos al arrancar. Verificar que las credenciales en `application-local.properties` coinciden con el usuario creado en PostgreSQL.

El backend tarda 30-60 segundos en responder en producción. El plan gratuito de Render suspende la instancia tras 15 minutos de inactividad. La primera petición la reanuda. Las siguientes responden con normalidad.

Error de CORS en el frontend. El backend solo acepta peticiones desde `https://nutri-fit-snowy.vercel.app` y `http://localhost:5173`. Si el frontend está en otra URL, actualizar `allowedOrigins` en `WebMvcConfig.java`.

Migration checksum mismatch al arrancar. Ocurre si se modifica un archivo de migración ya aplicado. Solución: añadir `spring.flyway.repair=true` en `application-local.properties` y arrancar una vez.

Anexo B — Referencia de endpoints de la API

El servidor escucha en `http://localhost:8080` en desarrollo y en `https://nutrifit-backend-ndoj.onrender.com` en producción. Todos los cuerpos usan `application/json`. Cualquier error devuelve `{ timestamp, status, error, message, path }`. Documentación interactiva en `/swagger-ui.html`.

B.1 Autenticación — `/api/auth`

Método	Ruta	Descripción	Éxito
POST	<code>/api/auth/register</code>	Registra usuario y abre sesión automáticamente	201
POST	<code>/api/auth/login</code>	Autentica usuario existente	200
POST	<code>/api/auth/logout</code>	Invalida el token de sesión activo	204

B.2 Alimentos — `/api/alimentos`

Método	Ruta	Descripción	Éxito	Errores
GET	<code>/api/alimentos</code>	Catálogo completo o filtrado por <code>?q=</code>	200	—
GET	<code>/api/alimentos/{id}</code>	Alimento por id	200	404
POST	<code>/api/alimentos</code>	Crea alimento	201	400
PUT	<code>/api/alimentos/{id}</code>	Actualiza alimento	200	400, 404
DELETE	<code>/api/alimentos/{id}</code>	Elimina alimento	204	404, 409

B.3 Escáner — `/api/escaner`

Método	Ruta	Descripción	Éxito	Errores
GET	<code>/api/escaner/{barcode}</code>	Busca alimento por código EAN en Open Food Facts	200	404

B.4 Comidas — `/api/comidas`

Método	Ruta	Descripción	Éxito	Errores
GET	<code>/api/comidas</code>	Comidas de un usuario por fecha	200	—
POST	<code>/api/comidas</code>	Crea comida	201	400

DELETE	/api/comidas/{id}	Elimina comida y sus ítems en cascada	204	404
GET	/api/comidas/{id}/items	Ítems con valores nutricionales calculados	200	404
POST	/api/comidas/{id}/items	Añade ítem a comida	201	400, 404
DELETE	/api/comidas/{cId}/items/{iId}	Elimina ítem de comida	204	404

B.5 Resumen diario y evaluación IA

Método	Ruta	Descripción	Éxito	Errores
GET	/api/resumen-diario	Totales nutricionales del día. Devuelve ceros si no hay comidas.	200	—
GET	/api/resumen/evaluacion-ia	Evaluación del resumen diario mediante IA	200	400, 503

B.6 Perfil — /api/perfil

Método	Ruta	Descripción	Éxito	Errores
GET	/api/perfil/{id}	Perfil con TMB y TDEE calculados	200	404
PUT	/api/perfil/{id}	Actualiza perfil; recalcula TMB y TDEE	200	400, 404

B.7 Ejercicios

Método	Ruta	Descripción	Éxito	Errores
GET	/api/ejercicios	Catálogo completo o filtrado por ?q=	200	—
GET	/api/ejercicios/{id}	Ejercicio por id	200	404
GET	/api/ejercicios-registro	Registros de un usuario por fecha	200	—
POST	/api/ejercicios-registro	Registra una sesión de ejercicio	201	400, 404
DELETE	/api/ejercicios-registro/{id}	Elimina un registro propio	204	404

B.8 Módulos adicionales

Método	Ruta	Descripción	Éxito
GET/POST/DELETE	/api/hidratacion	Registro diario de hidratación	200/201/204
GET/POST/DELETE	/api/peso-historial	Historial de peso corporal	200/201/204
GET	/api/tendencias	Tendencias nutricionales de los últimos 90 días	200

GET/POST	/api/plan-semanal	Plan de comidas generado por IA	200/201
GET/POST/DELETE	/api/lista-compra	Lista de la compra con sugerencias IA	200/201/204
GET	/api/retos	Catálogo de retos de fitness y progreso del usuario	200
GET	/api/gamificacion	Racha de días activos y puntuación acumulada	200
GET/PUT	/api/ia-config	Configuración del modelo de IA del usuario	200
GET	/api/health	Health check del backend	200